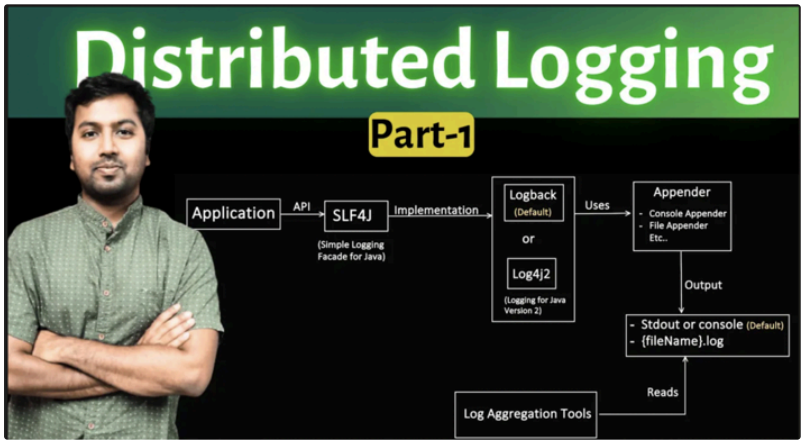
Distributed Logging - Part2

Pre-requisite: PART-1 of Distributed Logging



In previous part, we have seen the internal framework of Logger:

- What is SLF4j, Logback, Log4j2.
- Its Architecture (both High Level and Low Level).
- How to create Logger Object.
- Different Logger Levels and how Logger framework uses it.
- Parent-Child Hierarchy
  - ROOT Logger

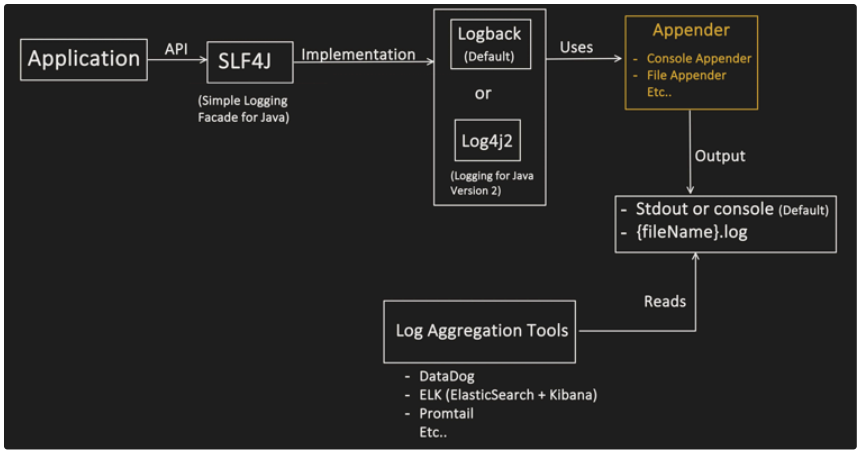
Concept && Coding

Lets start:

Appenders:

Appender is the component that decides where the log will go:

- console
- file
- DB
- Kafka etc.



Till now, we just created Logger Object and also seen how to override its Level through application.properties

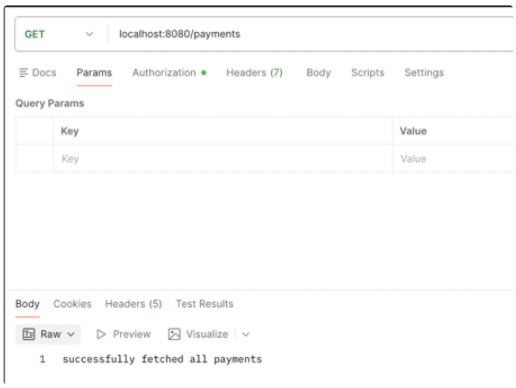
```
@RestController
public class PaymentController {

    Logger log = LoggerFactory.getLogger(PaymentController.class);

    @GetMapping("/payments")
    public String getPayments() {

        log.info("info log");

        return "successfully fetched all payments";
    }
}

```

Concept && Coding

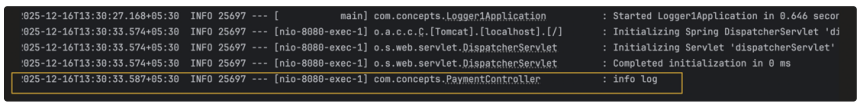
application.properties

```
logging.level.com.concepts.PaymentController=DEBUG

```

And when we ran our application, and invoked the endpoint:

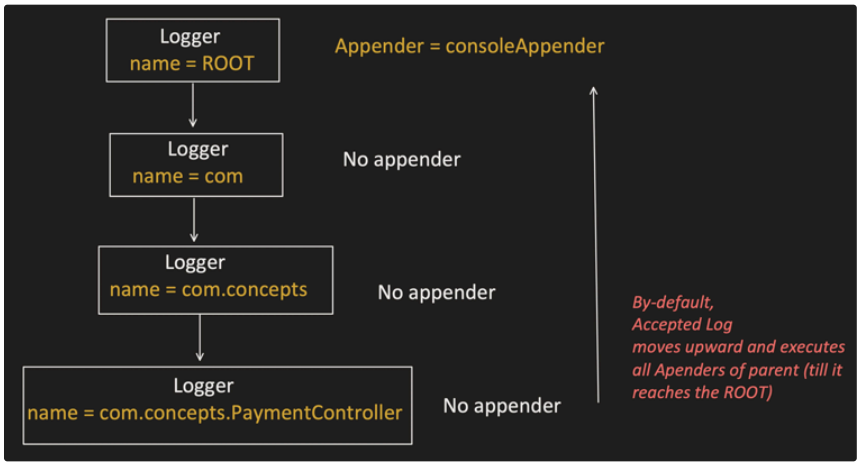


Logs are printed on Console properly:



As we know, Appender is the one that decides where the logs will go, and we have not added any Appender yet, so how its going to **CONSOLE**?

And in previous video we already answered the question:



Even though, when we have not defined any Appender for our Logger.

But by-default accepted Log propagate upwards till ROOT logger and executes all Appenders present in parent Loggers.

and ROOT Logger has default CONSOLE Appender present, which print the logs on CONSOLE.

Concept && Coding

<u>Now lets see, how to write Appenders for our Logger:</u>

Please Note:

Conceptually both Logback and Log4j2 use Appender to define where the logs will go, but they differs little in Syntax.

So, I am using Logback (which is Default for Spring boot)

src/main/resources/logback-spring.xml

This file contains info about:

- all appenders
- log Pattern
- all loggers
- root logger.

Syntax of logback-spring.xml

```
<configuration>

            <!-- 1. Define Appender, we can have more than 1 Appender -->
            <appender name="appender name" class = "fully.qualified.path.of.appender.class">
                    <encoder>
```

```
                            <pattern>%d %-5level %logger - %msg%n</pattern>
                            </encoder>
              </appender>

            <appender . . . >
                                . . .
             </appender>


            <!-- 2.  Loggers which refers above define Appender -->
            <logger name="logger name"  level="INFO/WARN/ERROR etc.." additivity="true/false">
                        <appender-ref ref="appender name"/>
            </logger>

            <logger  . . . >
                                . . .
            </logger>


            <!-- 3. Root (kind of fallback)  -->
             <root level= level="INFO/WARN/ERROR etc..">
                         <appender-ref ref="appender name"/>
             </root>

</configuration>

```

Encoder :

decides the format of the log.

Without Encoder, Appender does not know the log formatting and it will not work.

%d = date

Concept && Coding

%level = log statement level (INFO, WARN etc.)

%msg = log message

Etc..

will cover this Pattern in depth below.

Additivity :

- *controls appender propagation only.*
- *By default it is true.*
- *When log is accepted, it start running all the appenders of its Logger and parent loggers (till ROOT)*

If we don't want to execute parent appenders, we can make it false.

Level:

- *specify the level of the Logger.*
- *If missed, it will inherit the level from immediate parent.*
- *If immediate parent have also not defined the level, it will check its parent (if no found, ultimately will use ROOT logger level)*

Example: logback-spring.xml

```java
@RestController
public class PaymentController {

   //This will create 1 logger with name=com.concepts.PaymentController
    Logger log = LoggerFactory.getLogger(PaymentController.class);

    @GetMapping("/payments")
    public String getPayments() {

        log.info("info log");

        return "successfully fetched all payments";
    }
}

```

src/main/resources/logback-spring.xml

```xml
<configuration>

    <!-- Console Appender : Write logs to console -->
    <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">

        <!-- appender decides where the logs go, but its format is decided by encoder -->
        <encoder>
            <pattern>%d %-5level %logger - %msg%n</pattern>
        </encoder>

    </appender>

    <!-- currently we do not have any configuration for loggers, so all logs will go to root logger
    <logger ... >
        ...
    </logger>
    -->

    <!-- Root Logger -->
    <root level="INFO">
        <appender-ref ref="CONSOLE"/>
    </root>

</configuration>

```

Concept && Coding

I have not defined any configuration for Logger.

like I have one Logger with name="com.concepts.PaymentController"

so, I have not defined any thing for this logger like

- *which Appender to use?*
- *Logger Level?*

Etc.

So for appender, it propagates upward till ROOT (default behavior) and run their appenders.

Goes to:

1. *Logger with name = "com.concepts"*
   (config is not defined for this Logger, thus no appender present)

then

2. *Logger with name ="com"*
   (again config is not defined for this Logger too, thus no appender)

then

3. *ROOT Logger*
   we have defined only 1 appender i.e ConsoleAppender , so that will be used.

Also, we have not defined Logger level, so it will inherit the level from immediate parent, if they also don't have, it will go upward till ROOT

```
2025-12-16 16:28:55,265 INFO  org.apache.catalina.core.ContainerBase.[Tomcat].[localhost].[/] - Initializing Spring DispatcherServlet
2025-12-16 16:28:55,265 INFO  org.springframework.web.servlet.DispatcherServlet - Initializing Servlet 'dispatcherServlet'
2025-12-16 16:28:55,265 INFO  org.springframework.web.servlet.DispatcherServlet - Completed initialization in 0 ms
2025-12-16 16:28:55,277 INFO  com.concepts.PaymentController - info log

<pattern>%d %-5level %logger - %msg%n</pattern>
```

<u>Now, lets define the Appender for our Logger:</u>

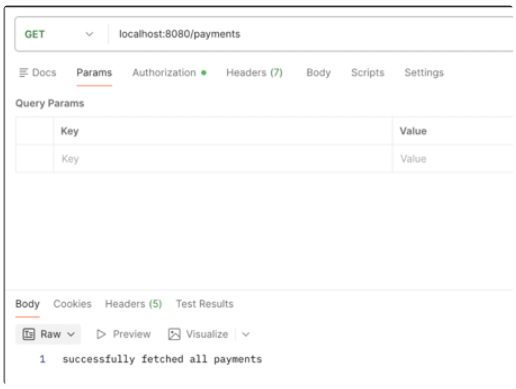
```java
@RestController
public class PaymentController {

    Logger log = LoggerFactory.getLogger(PaymentController.class);

    @GetMapping("/payments")
    public String getPayments() {

        log.info("info log");

        return "successfully fetched all payments";
    }
}

```

Concept && Coding

logback-spring.xml

```xml
<configuration>

    <!-- Console Appender : Write logs to console -->
    <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">

        <!-- appender decides where the logs go, but its format is decided by encoder -->
        <encoder>
            <pattern>%d %-5level %logger - %msg%n</pattern>
        </encoder>

    </appender>

    <logger name="com.concepts.PaymentController" level="INFO" additivity="true">
        <appender-ref ref="CONSOLE"/>
    </logger>

    <!-- Root logger -->
```

```
    <root level="INFO">
        <appender-ref ref="CONSOLE"/>
    </root>

</configuration>

```




```
2025-12-16 17:06:49,892 INFO  org.springframework.boot.web.servlet.context.ServletWebServerApplicationContext - Roo
2025-12-16 17:06:50,003 INFO  org.apache.coyote.http11.Http11NioProtocol - Starting ProtocolHandler ["http-nio-8080
2025-12-16 17:06:50,009 INFO  org.springframework.boot.web.embedded.tomcat.TomcatWebServer - Tomcat started on port
2025-12-16 17:06:50,013 INFO  com.concepts.Logger1Application - Started Logger1Application in 0.596 seconds (proces
2025-12-16 17:06:54,852 INFO  org.apache.catalina.core.ContainerBase.[Tomcat].[localhost].[/] - Initializing Spring
2025-12-16 17:06:54,853 INFO  org.springframework.web.servlet.DispatcherServlet - Initializing Servlet 'dispatcherS
2025-12-16 17:06:54,853 INFO  org.springframework.web.servlet.DispatcherServlet - Completed initialization in 0 ms
2025-12-16 17:06:54,867 INFO  com.concepts.PaymentController - info log
2025-12-16 17:06:54,867 INFO  com.concepts.PaymentController - info log
```

2 times it printed because:

```
<logger name="com.concepts.PaymentController" level="INFO" additivity="true">
            <appender-ref ref="CONSOLE"/>
</logger>

```

Concept && Coding

- *This Logger, has Console appender. So when log is accepted, its appender will get executed and it will print the log on Console.*

- *Additivity = true*
  Means, it will propagate upwards and run appenders of parents (till ROOT).

  ROOT logger also has ConsoleAppender, so that will get executed too and it will also print the log on console.

Making "additivity=false"

```
<configuration>

    <!-- Console Appender : Write logs to console -->
    <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">

        <!-- appender decides where the logs go, but its format is decided by encoder -->
        <encoder>
            <pattern>%d %-5level %logger - %msg%n</pattern>
        </encoder>

    </appender>

```

```
  <!-- additivity="false", Now for appender, it will not propagate upwards.But make sure that, the Logger has atleast 1 appender.-->
    <logger name="com.concepts.PaymentController" level="INFO" additivity="false">
        <appender-ref ref="CONSOLE"/>
    </logger>

    <!-- Root logger -->
    <root level="INFO">
        <appender-ref ref="CONSOLE"/>
    </root>

</configuration>

```

```
2025-12-16 17:13:49,681 INFO  org.springframework.boot.web.embedded.tomcat.TomcatWebServer - Tomcat started on port 8080
2025-12-16 17:13:49,685 INFO  com.concepts.Logger1Application - Started Logger1Application in 0.624 seconds (process runn
2025-12-16 17:13:54,758 INFO  org.apache.catalina.core.ContainerBase.[Tomcat].[localhost].[/] - Initializing Spring Dispa
2025-12-16 17:13:54,758 INFO  org.springframework.web.servlet.DispatcherServlet - Initializing Servlet 'dispatcherServlet
2025-12-16 17:13:54,758 INFO  org.springframework.web.servlet.DispatcherServlet - Completed initialization in 0 ms
2025-12-16 17:13:54,770 INFO  com.concepts.PaymentController - info log
```

When "additivity=false" is used, then make sure that, Logger has at least 1 appender. Else we get WARNING

```
<configuration>

    <!-- Console Appender : Write logs to console -->
    <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">

        <!-- appender decides where the logs go, but its format is decided by encoder -->
        <encoder>
            <pattern>%d %-5level %logger - %msg%n</pattern>
        </encoder>

    </appender>

    <logger name="com.concepts.PaymentController" level="INFO" additivity="false">
        <!-- <appender-ref ref="CONSOLE"/> -->
    </logger>

    <!-- Root Logger -->
    <root level="INFO">
        <appender-ref ref="CONSOLE"/>
    </root>

</configuration>

```

Concept && Coding

```
2025-12-16 17:17:21,744 INFO  org.springframework.boot.web.servlet.context.ServletWebServerApplicationContext - Root WebApplicationContext: initializati
2025-12-16 17:17:21,863 INFO  org.apache.coyote.http11.Http11NioProtocol - Starting ProtocolHandler ["http-nio-8080"]
2025-12-16 17:17:21,869 INFO  org.springframework.boot.web.embedded.tomcat.TomcatWebServer - Tomcat started on port 8080 (http) with context path '/'
2025-12-16 17:17:21,873 INFO  com.concepts.Logger1Application - Started Logger1Application in 0.612 seconds (process running for 0.735)
2025-12-16 17:17:24,406 INFO  org.apache.catalina.core.ContainerBase.[Tomcat].[localhost].[/] - Initializing Spring DispatcherServlet 'dispatcherServlet
2025-12-16 17:17:24,406 INFO  org.springframework.web.servlet.DispatcherServlet - Initializing Servlet 'dispatcherServlet'
2025-12-16 17:17:24,406 INFO  org.springframework.web.servlet.DispatcherServlet - Completed initialization in 0 ms
17:17:24,418 |-WARN in Logger[com.concepts.PaymentController] - No appenders present in context [default] for logger [com.concepts.PaymentController].
```

Another example:

Even when additivity is false (which is specifically for appender only)

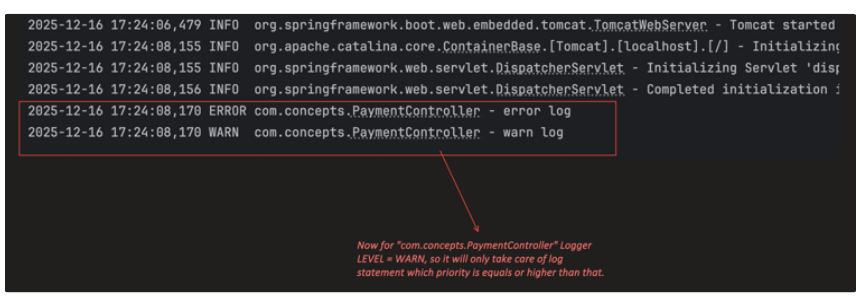
If Logger level is missing, it will always look upwards

```
@RestController
public class PaymentController {
```

```java

    Logger log = LoggerFactory.getLogger(PaymentController.class);

    @GetMapping("/payments")
    public String getPayments() {

        log.error("error log");
        log.warn("warn log");
        log.info("info log");
        log.debug("debug log");
        log.trace("trace log");

        return "successfully fetched all payments";
    }
}

```

```xml
<configuration>

    <!-- Console Appender : Write logs to console -->
    <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">

        <!-- appender decides where the logs go, but its format is decided by encoder -->
        <encoder>
            <pattern>%d %-5level %logger - %msg%n</pattern>
        </encoder>

    </appender>

    <logger name="com.concepts" level="WARN">
        <appender-ref ref="CONSOLE"/>
    </logger>

    <!-- Level is missing for "com.concepts.PaymentController",
        so it will look its immediate parent logger "com.concepts" and inherit its LEVEL (i.e. WARN in this case)
    -->
    <logger name="com.concepts.PaymentController" additivity="false">
        <appender-ref ref="CONSOLE"/>
    </logger>

    <!-- Root Logger -->
    <root level="INFO">
        <appender-ref ref="CONSOLE"/>
    </root>

</configuration>

```

Concept && Coding



Now lets see, what all different Appenders Logback framework provides:

- **Console Appender :**

  - Prints log to output stream called STDOUT (Standard Output)

- Local run = Terminal reads STDOUT content
- Docker = Docker Logging Driver reads STDOUT
- Cloud (Azzure) = Azzure Monitor collects STDOUT logs

- Platform handles Log collection, we don't have to worry.
- Fast
- No persistence (so no space issue)

```
<configuration>

    <!-- Console Appender : Write logs to console -->
    <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">

        <!-- appender decides where the logs go, but its format is decided by encoder -->
        <encoder>
            <pattern>%d %-5level %logger - %msg%n</pattern>
        </encoder>

    </appender>

    <!-- Root Logger -->
    <root level="INFO">
        <appender-ref ref="CONSOLE"/>
    </root>

</configuration>

```

```
2025-12-16 17:13:49,681 INFO  org.springframework.boot.web.embedded.tomcat.TomcatWebServer - Tomcat started on port 8080
2025-12-16 17:13:49,685 INFO  com.concepts.Logger1Application - Started Logger1Application in 0.624 seconds (process runn
2025-12-16 17:13:54,758 INFO  org.apache.catalina.core.ContainerBase.[Tomcat].[localhost].[/] - Initializing Spring Dispa
2025-12-16 17:13:54,758 INFO  org.springframework.web.servlet.DispatcherServlet - Initializing Servlet 'dispatcherServlet
2025-12-16 17:13:54,758 INFO  org.springframework.web.servlet.DispatcherServlet - Completed initialization in 0 ms
2025-12-16 17:13:54,770 INFO  com.concepts.PaymentController - info log
```

pt && Coding

2. **<u>File Appender :</u>**

- Persist Logs.
- Not safe for production.

```
@RestController
public class PaymentController {

    Logger log = LoggerFactory.getLogger(PaymentController.class);

    @GetMapping("/payments")
    public String getPayments() {

        log.info("info log");

        return "successfully fetched all payments";
    }
}

```

```
<configuration>

    <!-- Appender -->
    <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
        <encoder>
```

```
            <pattern>%d %-5level %logger - %msg%n</pattern>
        </encoder>
    </appender>

    <!--
      It will create a file "app.log" inside logs package.
      And keep on appending the log statements.
      append = true
      Logs are retained even when application is restarted.
      append=false
      Logs are appended but when application re-started "app.log" file will be cleared and start adding logs again.
      Remember this file will keep on growing, so using this in production should be avoided.
    -->
    <appender name="FILE" class="ch.qos.logback.core.FileAppender">
        <file>logs/app.log</file>
        <append>true</append>
        <encoder>
            <pattern>%d %-5level %logger - %msg%n</pattern>
        </encoder>
    </appender>

    <!-- This Logger, has 2 appenders associated with it.
      So Logs statement will go to Console + FILE.
      and
      Additivity = false, so parent appenders will not run.
    -->
    <logger name="com.concepts.PaymentController" level="INFO" additivity="false">
        <appender-ref ref="CONSOLE"/>
        <appender-ref ref="FILE"/>
    </logger>

    <!-- Root Logger -->
    <root level="INFO">
        <appender-ref ref="CONSOLE"/>
    </root>

</configuration>
```

Concept && Coding

Console:

```
2025-12-16 19:53:17,636 INFO  com.concepts.Logger1Application - Started Logger1Application in 0.604 seconds
2025-12-16 19:53:19,575 INFO  org.apache.catalina.core.ContainerBase.[Tomcat].[localhost].[/] - Initializing
2025-12-16 19:53:19,575 INFO  org.springframework.web.servlet.DispatcherServlet - Initializing Servlet 'disp
2025-12-16 19:53:19,575 INFO  org.springframework.web.servlet.DispatcherServlet - Completed initialization i
2025-12-16 19:53:19,587 INFO  com.concepts.PaymentController - info log
```

logs/app.log:

app.log

```
2025-12-16 19:53:19,587 INFO  com.concepts.PaymentController - info log
```

3. **Rolling File Appender / Log Rotation :**

- Persist Logs.
- Controlled file size
- Support different policy: Policy decides WHEN and HOW log files are rotated.

a). Time based Rolling policy:

%d{} - controls rotation frequency

%d{yyyy-MM-dd} : rotation happens Daily

%d{yyyy-MM-dd-HH} : rotation happens Hourly

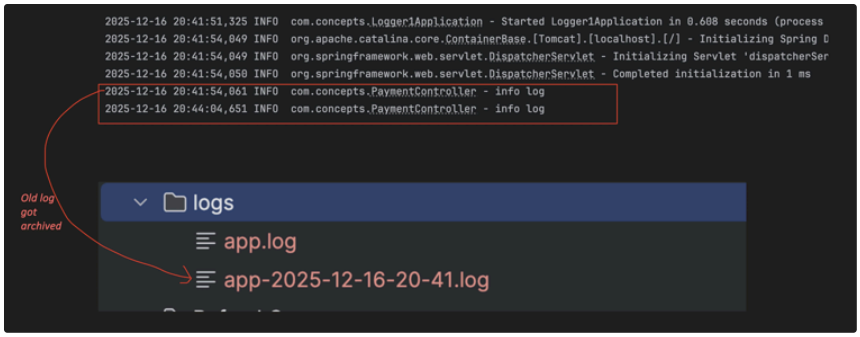
%d{yyyy-MM-dd-HH-mm} : rotation happens every minute

%d{yyyy-MM} : rotation happens Monthly

For testing purpose using %d{yyyy-MM-dd-HH-mm} policy

```
<appender name="ROLLING_FILE_TIME_BASED"
          class="ch.qos.logback.core.rolling.RollingFileAppender">

    <!--Current minute logging will be written here -->
    <file>logs/app.log</file>

    <rollingPolicy class="ch.qos.logback.core.rolling.TimeBasedRollingPolicy">
        <!--When archived.For each minute, we will be having different files -->
        <fileNamePattern>logs/app-%d{yyyy-MM-dd-HH-mm}.log</fileNamePattern>
        <!--
          Tells for how long we want to retain the file:
          - for minute wise, read it as, Last 30 minutes logs will be saved
          - for hourly wise, read it as, las 30 hrs logs will be saved
          - for day wise, read it as last 30days logs will be save
          etc.
        -->
        <maxHistory>30</maxHistory>
    </rollingPolicy>

    <encoder>
        <pattern>%d %-5level %logger - %msg%n</pattern>
    </encoder>

</appender>

<!-- Specific logger config -->
<logger name="com.concepts.PaymentController" level="INFO" additivity="false">
    <appender-ref ref="CONSOLE"/>
    <appender-ref ref="ROLLING_FILE_TIME_BASED"/>
</logger>

```

Concept && Coding



b). Size + Time based Rolling policy (BEST for production) :

```
<appender name="ROLLING_FILE_SIZE_TIME_BASED"
          class="ch.qos.logback.core.rolling.RollingFileAppender">

  <!--active file -->
    <file>logs/app.log</file>

    <rollingPolicy
class="ch.qos.logback.core.rolling.SizeAndTimeBasedRollingPolicy">

        <fileNamePattern>
            logs/app-%d{yyyy-MM-dd}.%i.log
        </fileNamePattern>
        <maxFileSize>100MB</maxFileSize>
        <maxHistory>30</maxHistory>
        <totalSizeCap>2GB</totalSizeCap>

    </rollingPolicy>

    <encoder>
        <pattern>%d %-5level %logger - %msg%n</pattern>
    </encoder>

</appender>
```

Concept && Coding

"maxFileSize" tells that 1 log file can not be greater than 100MB.

Let say, we are following day level logging and app.log file size increases more than 100MB: Logback closes the file, increment the index i (start from 0) and creates a new file.

app-2025-12-16.0.log (0-100MB)

app-2025-12-16.1.log (100-200MB)

.

.

.

But again, we need to limit the size which these combined (all) log file can use. That we can achieved through "totalSizeCap"

Writing our own Appender class:

When we want logs to go to custom destination like:

- DB
- Kafka

Then we have to write our own appender.

```java
/*
All appender class, need to extend this abstract class AppenderBase.

ILoggingEvent: Logback internal representation of a single log event/statement.

*/
public class DbAppender extends AppenderBase<ILoggingEvent> {

    @Override
    protected void append(ILoggingEvent event) {

        // Extract log data
        String message = event.getFormattedMessage();
        String loggerName = event.getLoggerName();
        long timestamp = event.getTimeStamp();

        // Custom destination logic
        // (Pretend this is a DB insert)
        System.out.println(
                "Saving ERROR log to DB: "
                        + timestamp + " - "
                        + loggerName + " - "
                        + message
        );
    }
}

```

Concept && Coding

logback-spring.xml

```xml
<appender name="DB_APPENDER"
          class="com.concepts.customappender.DbAppender">
    <!-- no encoder is needed here, all handled by java code, where to go and how -->
</appender>

<!-- Specific logger config -->
<logger name="com.concepts.PaymentController"
        level="INFO"
        additivity="false">
    <appender-ref ref="DB_APPENDER"/>
</logger>

```